Assignment 3: Exercise Recognition and Rep Counting

Overview

You will build a pipeline that takes one exercise recording and answers three questions: **where is the set** (segmentation), **which exercise is it** (classification, eight exercises), and **how many reps** (rep counting). This is your introduction to event-based sensing: in A2, every second of a recording was the same activity, but an exercise recording is rest, then a set of separate reps, then rest again.

Done in **groups of three**, using the class corpus from the Exercises Study.

Use of AI coding assistants (Claude Code, Cursor, Copilot, ChatGPT) is **permitted** on the code and the exploration report. The assignment quiz is closed-resource and graded on whether you understood what you were doing.

What's in this repo

```
├── RUBRIC.md                       # assignment instructions and grading
├── A3_exercise_recognition.ipynb   # student notebook with TODOs: segmentation, reps, features, models, failures
├── util.py                         # loading, filtering, baselines, windowing, evaluation (do not modify)
└── data/                           # class corpus as of Oct 26; 430 accelerometer CSVs
    ├── manifest.csv                # one row per recording: path, exercise, reps, placement
    ├── ab_crunches/                # 28 recordings
    ├── bicep_curls/                # 50 recordings
    ├── jumping_jacks/              # 87 recordings
    ├── lat_raises/                 # 19 recordings
    ├── lunges/                     # 42 recordings
    ├── push_ups/                   # 58 recordings
    ├── sit_ups/                    # 40 recordings
    └── squats/                     # 106 recordings
                                    # filenames: <number>_<recording date and time>.csv (Sensor Logger Accelerometer)
```

Data

A3 uses recordings from the **CS328 Exercises Study** you joined in A0. Each recording is **one set of one exercise**: stand still ~5 s → do the set → stand still ~5 s → Stop. Same sensors and CSV format as A1 and A2. Labels come from the questionnaire:

- **exercise** → the folder name (8 exercises)
- **reps** → the number of reps you counted; this is the ground truth for rep counting
- **placement** → where the phone was: Hand (200 recordings), Trouser (188), Jacket (23), Armband (13), Bag/backpack (6)

`data/manifest.csv` has one row per recording with these three labels and the path of the CSV. We removed recordings with a missing answer, fewer than 10 reps, a very small file (under 500 kB), or a duplicate upload. We also left out the five exercises with fewer than 20 recordings (bench press, lat pulldown, deadlifts, tricep extension, overhead press).

Things to know before you start (more in the notebook):

- **Gravity is removed**, as in A2, so the orientation of the phone is not in the data.
- **Not everyone stood still.** Many recordings start and end with **phone handling** (putting the phone in a pocket, taking it out to tap Stop), which looks like a short burst of large motion.
- **Placement matters.** With the phone in a trouser pocket, upper-body exercises (bicep curls, lat raises) barely move the phone.

- **Reps are self-reported.** Most are right, but some may be off by a rep or two.
- **Some classes are small**: lat raises has 19 recordings and ab crunches 28, while squats has 106.

Analysis

Your job is to build a **three-step pipeline** that takes a recording, finds the set, recognizes the exercise, and counts the reps.

Three design axes A3 has **three design choices**. Treat them separately in the report.

Axis	What you choose	Example knobs
Segmentation	How to find the set inside the recording	Fixed threshold vs adaptive threshold (set from the recording's own quiet and active levels), energy window length, ignoring short bursts (phone handling), trimming the still parts that the protocol asks for
Classification	Which features per window + which model	Window length, per-axis features (which axis moves), dominant frequency (rep rate), correlation between axes; Decision Tree, Random Forest, kNN, ...; majority vote over windows
Rep counting	How to turn the set into a number	Which signal (magnitude, one axis, or the direction that moves the most), low-pass cutoff, minimum distance between peaks, peak prominence, exercise-specific settings

Errors compound: if the segmentation keeps the phone handling, the rep counter counts it as reps, and the classifier sees windows that are not the exercise. The report asks you to say which step caused each error.

Required: compare at least 2 segmentation approaches and at least 2 models

- **Segmentation**: compare the provided baseline (fixed threshold) with **at least one approach of your own**, and measure the effect on rep counting and on classification.
- **Models**: compare **at least two different classifier types** (e.g., Decision Tree vs Random Forest) on the same feature set.

Train / test split All groups use the same split: `split_recordings` in util.py puts 70% of the recordings in train (301) and 30% in test (129), with the same mix of exercises in both. The split is by **recording**, not by window: windows from the same recording are very similar, so if they end up in both train and test, the accuracy looks much better than it really is. Tune

everything (segmentation, window length, features, hyperparameters, rep-counting settings) on the training recordings, and use the test recordings only for the final numbers. The validation set inside the training recordings is small (76 recordings), so its accuracy can change by several points depending on which recordings end up in it. Do not treat a difference of a few points as a real finding.

Deliverables

Submit a zip file on Canvas by **Nov 1, 11:59 PM**, one submission per group:

1. completed `A3_exercise_recognition.ipynb` notebook and any potential .py files generated by you.

2. `exploration_report.pdf` — your systematic exploration (the depth artifact) Use the templates below.

§1: Segmentation exploration

For each segmentation approach you tried (test recordings, with your final rep counter):

Approach	Key parameter(s)	Rep MAE	Within ±2 reps	Notes
(e.g., Fixed threshold, baseline)	moving std > 0.5 m/s²	4.2	47%	Keeps the phone handling at both ends
(e.g., Adaptive threshold + drop short bursts)	25% between quiet and active level, bursts < 3 s dropped	3.6	55%	Cuts the first reps when the person did not stand still
…	…	…	…	…

One paragraph: which approach won and why? Where did each one break? Did your segmentation change the classification accuracy (see the feature exploration table in the notebook)? Include one figure of a recording where the baseline fails.

§2: Classification exploration

Which features did you try (feature exploration in the notebook)? Which exercises did they separate? Aim for ~5-10 features of your own on top of the 7 baseline features; the table can show more that you tried and dropped.

Feature	Exercises it helps separate	Effect on accuracy (+/− / no change)	Rationale
(e.g., dominant frequency)	jumping jacks vs squats	+5%	Jumping jacks are about twice as fast as squats
(e.g., std of x, y, z)	bicep curls vs lunges	+3%	Different exercises move the phone along different axes
…	…	…	…

For each model you tried, on the same features:

Model	Key hyperparameter(s)	Window accuracy	Recording accuracy	Notes
(e.g., Decision Tree)	max_depth=5, entropy	38%	41%	Baseline, interpretable
(e.g., Random Forest)	n_estimators=200	47%	52%	Gains came from push-ups and lunges
…	…	…	…	…

One paragraph: which won and why? Did the better model help all exercises equally, or did the gain concentrate on specific exercises?

§3: Rep counting exploration

For each rep counter you tried (test recordings, with your final segmentation):

Approach	Signal	Key parameter(s)	Rep MAE	Within ±2 reps	Notes
(e.g., Baseline peaks)	filtered magnitude	height > mean, ≥ 1 s apart	7.0	25%	Two peaks per rep for squats; counts the phone handling
(e.g., Peaks on the most active axis)	x, y, or z with the largest std, low-pass 1.5 Hz	distance = 0.6 × rep period	4.0	48%	Fails when the phone barely moves
…	…	…	…	…	…

One paragraph: what made the biggest difference? Which exercises and placements are hardest to count, and why?

§4: Final pipeline + confusion matrix + error attribution

- The exact segmentation, window length, feature set, classifier + hyperparameters, and rep counter.
- The **recording-level** confusion matrix of your final pipeline on the test recordings (8 × 8; embed the figure from the notebook).
- For each off-diagonal cell that is meaningfully nonzero:
  - One sentence: what caused this confusion?
  - Attribute it to one of the three: **segmentation** (the segment is wrong: phone handling kept or reps cut off), **feature side** (the features do not separate these exercises), or **model side** (the features separate them, but the model fails, e.g., because the class is small).

§5: Top failure modes

For each of your **top 2 failure modes** (a confused exercise pair, or a rep-counting failure):

- What fails (e.g., lat raises predicted as squats; push-ups with the phone in a pocket counted too many times).

- A specific recording where the failure is clear. Embed a figure showing the trace with your segment (and your detected reps, for a rep-counting failure).
- 1-2 sentences: the physical mechanism causing the failure.
- 1 sentence: is this primarily a **segmentation**, **classification** (feature or model), or **rep counting** problem?
- 1 sentence: one specific change that would help.

§6: Contribution statement

- One line: who did what.

Assessment

Total: 48 points. Mix of group-level (artifact) and individual (assignment quiz and data collection). The general concept quiz on segmentation and exercise recognition is graded separately as part of the class quiz series.

Individual-level (24 pts)

- **Assignment Quiz (12 pts)**
- **Data Collection (12 pts)**

Group-level (24 pts)

- **Code grading (12 pts)**
- **Exploration report + contribution statement (12 pts)**

On using AI

You may use AI coding assistants on the notebook and the exploration report. The assignment quiz tests whether you, specifically, engaged with the work. Questions on the quiz directly reference specific sections of your group's submitted exploration report — vague answers that don't match the report will be flagged.

Late Policy

- Each group has **5 late days**, usable across all assignments for final code submissions.
- Late submission penalty: −10% per day after the 5 free late days.
- **Assignment quiz is individual; no late days apply** except for documented emergencies.
- Significant contribution imbalance (as reported) → individual grading may be triggered.

FAQ

Q: Can I use ChatGPT/Claude to write the notebook and the exploration report? Yes. You're being assessed on understanding, not typing. The quiz catches faking.

Q: Can I use deep learning (CNN, LSTM, etc.)? Yes, if you want (for example, a small 1D-CNN on raw windows). Hand-crafted window features + classical ML is the expected path and works well at this scale. With ~430 recordings, a neural network can easily overfit. If you try one, document the tradeoff in §2.

Q: Why do we split by recording and not by window? Windows from the same recording come from the same person, phone, and placement, so they look almost the same. If some of them are in train and some in test, the model is tested on data it has basically already seen, and the accuracy is too optimistic. `split_recordings` keeps every recording entirely in train or entirely in test.

Q: Why does the baseline rep counter count too many reps? Two reasons. First, the magnitude often has two peaks per rep: going down and coming up both accelerate the phone. Second, the baseline segmentation keeps the phone handling at the start and end, and those bursts become extra peaks. Reps are slow (one every 1-4 s), so a stronger low-pass filter, one axis instead of the magnitude, and a minimum distance between peaks all help.

Q: The baseline counts jumping jacks about twice with the phone in my pocket. Why? One jumping jack has two jumps (feet out, feet in), so the hip moves up and down twice per rep, while the arms (phone in hand) go up and down once. The baseline counter sees both jumps. A low-pass filter around 1 Hz mostly merges them into one cycle per rep. Note that `count_reps` does not get the placement, so any fix has to work from the signal itself.

Q: Some recordings are almost flat. Is my code broken? Probably not. If the phone is on a part of the body that does not move (e.g., bicep curls with the phone in a trouser pocket, or an armband on the other arm), there is little to see. A few recordings have no motion at all, not even phone handling. The whole trace is noise of about 0.01 m/s^2, so the phone was most likely not on the body. This is a data collection error, and no method can classify or count these recordings. Expect them to fail and discuss them in §5, instead of tuning your pipeline for them.

Q: Why are so many exercises predicted as squats? Squats is the largest class (106 recordings) and some classes are small (lat raises has 19). A model that is unsure picks the class it saw most. Sit-ups and ab crunches are a common case: with gravity removed, lying down and standing up look alike in the data. Ab crunches are the hardest, so expect to get almost none of them right. Try `class_weight='balanced'` (where the model supports it) and explain what you see in §4.

Q: My count is off, but the plot looks right. What should I do? Count the reps in the plot yourself. Reps are self-reported, so a few labels may be wrong. If you find one, you can use it as an example in §5.

Q: My segmentation barely changes the results. Did I do something wrong? No. A rep counter with a strong low-pass filter and a minimum distance between peaks already ignores much of the phone handling, and so does a random forest. So a better segment may change the numbers only a little. That is a fine finding. Report it and explain why in §1.

Q: What window length should I use? Try several. A window should contain at least one full rep (one rep takes 1-4 s), so very short windows may not see the whole movement. Often the differences between window lengths are smaller than the validation noise, and that is a fine finding too. Report what you tried in §2.